

Kumo Development Roadmap

Project: Kumo — Collaborative Travel Super-App

Current Version: 2.0 (as-built)

Previous Version: 1.0 (original plan)

Target Launch: Q3 2026

What Changed: v1.0 → v2.0

The original 5-stage plan was restructured during implementation. Stages were broken into smaller, shippable chunks. Several planned features were descoped or simplified; others not in the original plan were added based on what made the product more useful immediately.

Summary of Divergences

Area	v1.0 Plan	v2.0 Actual	Reason
Offline storage	Isar database (full offline-first)	SharedPreferences JSON cache	Isar adds codegen complexity with marginal benefit for this scope
Conflict resolution	Vector clocks + event sourcing	Last-write-wins on JSONB	Vector clocks are production-scale infrastructure; deferred
AI tier	Skeletal mode + Concierge mode (CrewAI agents, streaming)	Single-call generation (Claude Haiku)	Agentic AI requires backend infrastructure not yet in place
AI security	Supabase Edge Function (key server-side)	Direct client call (key in .env)	Edge function scaffolding deferred; acceptable for dev/beta
Fintech	Expense split + Virtual Debit Card (Stripe Issuing) + Stripe Connect	Expense split only	Stripe Issuing requires legal/compliance review; deferred
Social layer	Feed with likes, comments, follows, gamification (XP/badges)	Read-only Discover feed (public itineraries)	Full social graph is a separate product surface; deferred
B2B portal	Multi-tenant schema, travel policies, admin dashboard	Not built	Requires pilot customer; premature to build without validated demand

Area	v1.0 Plan	v2.0 Actual	Reason
Chat polish	Typing indicators, read receipts	Not built	Core messaging shipped; polish deferred to Stage 9
New (not in v1.0)	—	Packing lists, trip notes, share sheet, offline banner, home search, profile stats	Surfaced as higher value during implementation

Stage Mapping: v1.0 → v2.0

v1.0 Stage 1 (MVP Core) (subset only)	→	v2.0 Stage 1 (Foundation) ✓ v2.0 Stage 7 (Offline Cache) ✓
v1.0 Stage 2 (Collaboration) (simplified)	→	v2.0 Stage 2 (Collaboration) ✓
v1.0 Stage 3 (Agentic AI) (simplified)	→	v2.0 Stage 3 (AI Generation) ✓ v2.0 Stage 8 (AI Integration) ✓
v1.0 Stage 4 (Fintech + Ratings) (no Stripe)	→	v2.0 Stage 4 (Expenses + Ratings) ✓
v1.0 Stage 5 (Social + B2B) (subset only)	→	v2.0 Stage 6 (Discover + Notes) ✓
Not in v1.0	→	v2.0 Stage 5 (Packing Lists) ✓
Not in v1.0	→	v2.0 Stage 7 (Connectivity + Search)
✓ (partial)		
Not in v1.0	→	v2.0 Stage 9 (Polish + Release Prep)
■		

v2.0 Stages (As Built)

Stage 1 — Foundation ✓

Shipped: Auth (sign up, login, password reset, session persistence), itinerary CRUD, clean architecture (Domain → Data → Presentation), Riverpod state management, GoRouter navigation, Supabase integration, Material 3 theme, error/failure hierarchy.

vs. v1.0: Matches the v1.0 Stage 1 scope except Isar offline DB (replaced by SharedPreferences in Stage 7) and optimistic UI (deferred).

Stage 2 — Real-Time Collaboration ✓

Shipped: Trip members (JSONB array), role management (owner/editor/viewer), invite by email (Supabase lookup), role change + member removal, real-time chat (Supabase Realtime), inbox with unread badge, privacy settings.

vs. v1.0: No vector clocks, event sourcing, or conflict resolution. Concurrent edits are last-write-wins on the `itineraries` JSONB `members` column. No typing indicators or read receipts.

Stage 3 — AI Itinerary Generation

Shipped: `AiGenerationRequest` entity with destination, dates, travel style, interests. Single Claude Haiku call via Anthropic API. JSON parsing with fallback regex extraction. Bottom sheet UI with form → loading → preview → accept/regenerate flow. Integrated into Create Trip page.

vs. v1.0: No Concierge mode, no CrewAI/LangChain agents, no streaming responses, no Supabase Edge Function (API key is client-side in `.env`), no generation history or caching, no iterative refinement ("add more nightlife"). This covers roughly the Skeletal Mode from v1.0.

Stage 4 — Expense Splitting + Ratings

Shipped: Expense logging (title, amount, category, payer, per-member splits), budget tracker with progress bar, settlement calculation (greedy debt minimization), real-time expense stream. Star ratings (1–5) with comments on activities/destinations, rating summary view.

vs. v1.0: No virtual debit card, no Stripe Issuing or Stripe Connect, no payment settling, no CSV export, no PCI-DSS compliance work. The "Splitwise Clone" scope shipped; the fintech layer did not.

Stage 5 — Packing Lists

Not in v1.0. Added: per-trip packing list with Supabase Realtime sync, checkbox toggle (shared state — all members see changes live), progress bar, category field, per-item creator tracking, delete with swipe. Integrated as a tab in the trip detail page.

Stage 6 — Discover Feed, Trip Sharing & Notes

Shipped: `is_public` toggle on itinerary (owner-only), Discover tab showing public trips with search, clone itinerary (new trip with fresh owner/members). Native share sheet via `share_plus`. Shared trip notes (full-screen debounced editor, real-time sync, read-only for viewers).

vs. v1.0: This is a lightweight version of v1.0 Stage 5's social/discovery concept — no likes, comments, follows, or social graph. The Discover feed is read-only browsing + clone, not a full social feed.

Stage 7 — Connectivity, Offline Cache & Home Search

Shipped: `connectivity_plus` network monitoring, `isOnlineProvider` Riverpod stream, `OfflineBanner` shown in shell when offline, `ItineraryLocalDataSource` (SharedPreferences JSON cache), `fetchItineraries` falls back to cache on failure. Home page search bar (client-side filter on title/description). Avatar tap fixed to `/profile`.

vs. v1.0: This delivers the offline-fallback part of v1.0 Stage 1 but with SharedPreferences instead of Isar. No auto-sync of offline mutations (reads only).

Stage 8 — AI Integration into Existing Trips + Profile Stats

✔

Not in v1.0 as a separate stage. Added "AI" button (owner/editor only) in the trip detail Schedule header — opens the generation sheet and appends items to the existing trip. Profile stats card showing total trips, upcoming trips, days traveled derived from `itineraryListProvider`.

Stage 9 — Polish, Onboarding & Release Prep

Not yet started.

Planned scope:

- First-run onboarding (3-screen walkthrough, permission requests)
- App icon + launch screen
- Chat polish: typing indicators, read receipts
- Transition animations (Hero, page transitions)
- Accessibility audit (semantic labels, contrast)
- `flutter test` suite (unit tests for domain usecases)
- `flutter analyze` zero-warning enforcement in CI
- Supabase Edge Function for Anthropic API key (move key server-side before public launch)

What Remains from v1.0 (Deferred / Not Implemented)

These items from the original roadmap were explicitly descoped. They represent the gap between the current build and the full v1.0 vision.

Deferred to a future version

Feature	Original Stage	Why Deferred
Vector clocks + conflict resolution	S2	Infrastructure complexity; needs dedicated engineering sprint
Chat: typing indicators, read receipts	S2	Core messaging shipped; polish can follow
Supabase Edge Function for AI key	S3	Acceptable risk for beta; required before App Store release
Concierge AI mode (agents, streaming)	S3	Requires backend agent infrastructure (LangChain/CrewAI)
Generation history + caching	S3	Low priority until usage data confirms demand
Virtual Debit Card (Stripe Issuing)	S4	Legal/compliance review required; months of lead time
Stripe Connect (user-to-user payments)	S4	Dependent on virtual card work

Feature	Original Stage	Why Deferred
CSV expense export	S4	Low effort; add to Stage 9 polish
Social feed (likes, comments, follows)	S5	Full social graph is a separate product surface
Gamification (XP, achievements, badges, leaderboards)	S5	Engagement mechanic; valid only after retention baseline is established
B2B portal (multi-tenant, travel policies, admin dashboard)	S5	Requires pilot customer; no validated demand yet

Timeline Overview (v2.0 Actual)

Jun 2026

— Stage 1: Foundation

— Stage 2: Collaboration

— Stage 3: AI Generation

— Stage 4: Expenses + Ratings

— Stage 5: Packing Lists

— Stage 6: Discover + Notes + Sharing

— Stage 7: Connectivity + Offline

— Stage 8: AI Integration + Profile

— Stage 9: Polish + Release Prep

✓ Weeks 1–2

✓ Week 3

✓ Week 3

✓ Week 4

✓ Week 4

✓ Week 5

✓ Week 5

✓ Week 5

Upcoming

Quality Gates (Carried Over from v1.0)

- flutter analyze passes with zero issues — enforced at every stage ✓
- No secrets committed to git (.env in .gitignore) ✓
- Supabase RLS enabled on all tables with tested policies ✓
- Clean Architecture layer boundaries respected (no presentation → data imports) ✓
- API calls < 2s target; UI load < 1s target

Gaps vs. v1.0 quality gates:

- **Test coverage:** No automated test suite yet — planned for Stage 9
- **Accessibility:** No formal WCAG 2.1 audit yet — planned for Stage 9
- **Code review:** Solo development; no PR review process established

End of Development Roadmap v2.0